UNIT-4

Structures in C:

We cannot use arrays if we want to represent a collection of data items of different types using a single name.

C supports a constructed data type, known as "structures" for storing different types of data into a single unit.

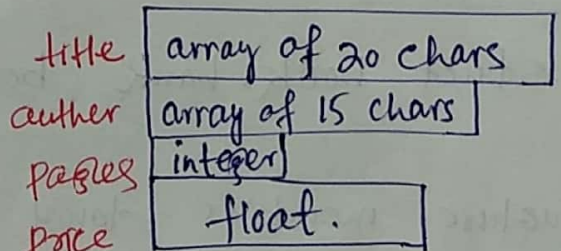
Defining a structure:

The general format of a structure is as follows:

```
struct tag_name
{
    data-type member1;
    data-type member2;
        :
};
```

Example:

```
struct book_bank
{
    char title[20];
    char auther[15];
    int pages;
    float price;
};
```



(Structure template)

Scanned by CamScanner

The keyword struct is used for defining a structure. Here book-bank is called a structure name or structure tag.

The tag name such as book-bank can be used to declare structure variable of its type.

The structure definition must be terminated with a semicolon.

The definition of a structure doesnot declare any variables. It simply describes a format called template to represent information.

Declaring a structure variable :

A structure variable declaration is similar to declaration of variable of any other type.

It includes keyword struct, structure tag name, list of variable name separated by commas and termination semicolon.

Example :

```
struct book-bank book1, book2, book3;
```

structure members donot occupy any memory until they are associated with the structure variables such as book1.

Scanned by CamScanner

It is also allowed to combine both the structure definition and variable declaration in one statement.

```
struct book_bank
{
    char title[20];
    char auther[15];
    int pages;
    float price;
} book1, book2, book3;
```

Accessing structure members :

We can access the structure variable using member operator (.) with structure variable. (Also called dot operator)

Example :

```
#include<stdio.h>
struct book_bank
{
    char title[20];
    char auther[15];
    int pages;
    float price;
};
int main()
{
    struct book_bank book;
    printf("Enter the book details:");
    scanf("%s %s %d %f", book.title, book.auther,
                &book.pages, &book.price);
    printf("%s %s %d %f", book.title, book.auther,
                book.pages, book.price);
    return 0;
}
```

Scanned by CamScanner

Structure Initialization :

A structure can be initialized as follows :

```
void main()
{
    struct student
    {
        int rollno;
        int marks;
        char name[20];
    };
    struct student s1 = { 101, 420, "Shyam" };
    struct student s2 = { 104, 255, "Binay" };
      :
}
```

The initialization of a structure variable must have the following elements.

- The keyword struct,
- Structure tag name,
- name of the variable to be declared,
- The assignment operator,
- A set of values for the members of the structure variable, separated by commas and enclosed in braces.
- A terminating Semicolon.

Rules for Initializing structures :

we cannot initialize individual members inside structure defn.
The order of values in the braces must match the order of members in the structure definition.
Partial initialization is allowed. We can initialize first few members.
The uninitialized members will be assigned with default values like : - zero is assigned for int and float values
- '\0' for character strings.

Scanned by CamScanner

Copying and Comparing structure variables :

Two variables of the same structure can be copied the same way as ordinary variables.

```
struct student S1, S2;
    S1 = S2;
```

However the statements like S1 == S2 and S1 != S2 are not permitted.

C doesnot permit any logical operations on structure variables. In case we required we can do by comparing members individually.

Example:

```
#include<stdio.h>
struct class
{   int rollno;
    char name[20];
    float marks;
};
int main()
{    int x;
    struct class S1 = { 11, "Ram", 520.5 };
    struct class S2 = { 22, "Shyam", 167.3 };
    struct class S3;
    S3 = S2;
    x = ((S3.rollno == S2.rollno) &&
         (S3.marks == S2.marks)) ? 1 : 0;
  if (x == 1)
  {    printf (" Student2 and Student3 are same \n");
       Printf (" %d %s %f", S3.rollno, S3.name, S3.marks);
  } else
       print(" \n Student2 and Student3 are
                                different \n \n");
} return 0;
```

Scanned by CamScanner

Array of structures :

we can declare an array of structure as follows :

```
struct class info[100];
```

here each element of array info is of type structure class.

Example :

```
#include <stdio.h>
struct marks
{
    int sub1;
    int sub2;
    int sub3;
    int total;
};
int main()
{
    int i;
    struct marks s[3] = {{45,67,80,0}
                         {57,68,50,0}
                         {36,45,89,0}};
    struct marks sum = {0,0,0,0};
for (i=0; i<=2; i++)
{
    s[i].total = s[i].sub1 + s[i].sub2 + s[i].sub3;
    sum.sub1 = sum.sub1 + s[i].sub1;
    sum.sub2 = sum.sub2 + s[i].sub2;
    sum.sub3 = sum.sub3 + s[i].sub3;
    sum.total = sum.total + s[i].total;
}
  printf("Students Total \n\n");
for (i=0; i<3; i++)
    printf ("Student[%d] = %d \n", i+1, s[i].total);
printf ("\n Subject total \n\n");
```

Scanned by CamScanner

```
    printf("%s %d \n %s %d \n %s %d \n",
            "Subject1", sum.sub1, "Subject2", sum.sub2,
            "Subject3", sum.sub3);
    printf("\n Grand total = %d", sum.total);
}
```

Arrays within Structures :

```
#include<stdio.h>
struct marks
{
    int sub[3];
    int total;
};
int main()
{
    struct marks S[3] = {45,67,81,0,75,53,69,0,57,36,71,0};
    struct marks sum = {0,0,0,0};
    int i,j;
    for(i=0; i<=2; i++)
    {
        for(j=0; j<=2; j++)
        {
            S[i].total = S[i].total + S[i].sub[j];
            sum.sub[j] = sum.sub[j] + S[i].sub[j];
        }
        sum.total = sum.total + S[i].total;
    }
    printf("Student Total \n \n");
    for(i=0; i<=2; i++)
        printf("Student[%d] %d \n", i+1, S[i].total);
    printf("Subject Total \n \n");
    for(j=0; j<=2; j++)
        printf("Subject %d %d \n", j+1, sum.sub[j]);
    printf("\n Grand Total = %d \n", sum.total);
    return 0;
}
```

Scanned by CamScanner

Structure and Functions :

There are three methods by which the value of structure can be transfered from one function to another.

1. Pass each member of the structure as actual argument.
2. Pass a copy of entire structure to the called function.
3. Passing a pointer to structure.

The general format of sending a copy of structure to the called function is :

```
function name (structure variable name);
```

The called function must declare the structure type to hold and return structure variable.

When a function returns a structure, it must be assigned to a structure identical type in the calling function.

Example: Passing a copy of entire structure.

```
#include<stdio.h>
struct stores
{
    char name[20];
    float price;
    int quantity;
};
struct stores update(struct stores product, float p, int q);
float mul(struct stores stock);
```

Scanned by CamScanner

```
int main()
{   float pincr, value;
    int qincr;
    struct stores item = { "XYZ", 25.75, 12};
    printf ("Input increment values");
    scanf ("%f %d", &pincr, &qincr);
    item = update (item, pincr, qincr);
    printf ("updated values are:");
    printf ("Name : %s\n", item.name);
    printf ("Price : %f\n", item.price);
    printf ("Quantity : %d\n", item.quantity);
     value = mul (item);
    printf ("Value of the item = %f", value);
    return 0;
}
struct stores update (struct stores product,
                      float p, int q)
{
     product.price += p;
     product.quantity += q;
     return (product);
}
float mul (struct stores stock)
{    return (stock.price * stock.quantity);
}
```

Scanned by CamScanner

Pointer and Structures :

We can use pointers to store address of structure and access the members of structures.

To access the structure member using pointer we can take the help of arrow operator (→).

Example :

```
#include <stdio.h>
struct Book
{
    char name[20];
    int bookno;
    float price;
};
int main()
{
    struct Book b1 = { "C", 101, 150.00 };
    struct Book *ptr;
    ptr = &b1;
    printf("Name : %s\n", ptr→name);
    printf("Book No : %d\n", ptr→bookno);
    printf("Book price : %f\n", ptr→price);
    return 0;
}
```

Scanned by CamScanner

Nested Structures :

Structure within a structure is called nested structure.

```
#include <stdio.h>
struct address
{
    char city[10];
    int pin;
    int phone;
};
struct student
{
    char name[20];
    struct address add; // nested structure.
};
int main()
{
    struct student std;
    printf("Enter the student information");
    scanf("%s %s %d %d", std.name, std.add.city,
          &std.add.pin, &std.add.phone);
    printf("printing the student information");
    printf("name: %s \n City: %s \n Pincode: %d \n
          Phone: %d", std.name, std.add.city,
          std.add.pin, std.add.phone);
    return 0;
}
```

Alternate way :

```
struct student
{
    char name[20];
    struct address
    {
        char city[10];
        int pin;
        int phone;
    } add;
};
```

Scanned by CamScanner

Self Referential Structures :

Self referential structures are those structures that have one or more pointers which points to the same type of structure as their member.

```
#include <stdio.h>
struct node
{   int data1;
    int data2;
    struct node * link;
};
int main()
{   struct node obj1,obj2;
    obj1.link = NULL;
    obj1.data1 = 10;
    obj1.data2 = 20;

    obj2.link = NULL;
    obj2.data1 = 30;
    obj2.data2 = 40;
    obj1.link = &obj2;

    printf("%d", obj1.link->data1);
    printf("%d", obj1.link->data2);
    return 0;
}
```

Scanned by CamScanner

Unions

Union is a user defined data type similar to structure but the difference is that all members of union share the same memory location.

Union may contain many members of different type but it can handle only one member at a time and it is called active data member.

```
#include <stdio.h>
union emp
{
    float salary;
    int empno;
};
struct employee
{
    float salary;
    int empno;
};
int main()
{   union emp e;
    struct employee se;
    e.salary = 12500.00;
    e.empno = 101;
    printf ("Salary = %f \n", e.salary);  // 0.0
    printf ("Employee Number = %d", e.empno);  // 101
    printf ("Size of union = %d bytes", sizeof(e));  // 4
    printf ("Size of structure = %d bytes", sizeof(se));  // 6 or 8
}
```

Scanned by CamScanner

Structure within Union :

```
#include<stdio.h>
struct student
{
    char name[30];
    int roll;
    float percentage;
};
union example
{
    struct student std;
};
int main()
{
    union example ex1;
    printf("Enter the student details");
    scanf("%s %d %f", ex1.std.name,
          &ex1.std.roll, &ex1.std.percentage);
    printf("\nThe student details are\n");
    printf("Name = %s", ex1.std.name);
    printf("Roll No = %d", ex1.std.roll);
    printf("\n Percentage = %f", ex1.std.percentage);
    return 0;
}
```

Scanned by CamScanner

Bit Fields :

- There are situations where data item requires much less than 16 bit space. In such cases, we waste memory space.
- C permits us to use small bit fields to hold data items in a single word memory.
- Bit fields are used with structures and unions. We can specify size in bits of structure and union members.
- The general form of bit-field definition is :

```
struct tag-name
{
    data-type name1 : bit-length;
    data-type name2 : bit length;
        :
        :
    data-type nameN : bit-length;
};
```

Example :

```
#include <stdio.h>
struct date
{
    unsigned int d: 5;
    unsigned int m: 4;
    unsigned int y;
};
```

Scanned by CamScanner

```
int main()
{
    printf("Size of date is %lu bytes\n", sizeof(struct date));
    struct date dt = {31, 11, 2019};
    printf("Date is %d/%d/%d", dt.d, dt.m, dt.y);
    return 0;
}
```

output

```
Size of date is 8 bytes
Date is 31/11/2019
```

There are several specific points to observe:

1. The first field always starts with the first bit of the word.
2. The sum of lengths of all the fields in a structure should not be more than the size of a word. In case it is more, the overlapping field is automatically forced to beginning of the next word.
3. There can be unnamed fields declared with size.
   ```
   unsigned : bit-length;
   ```
   Such fields provide padding within the word.
4. There can be unused bits in the word.
5. We cannot take the address of a bit field variable. So we cannot use scanf to read values into bit fields.
6. We cannot use pointers to access the bit fields.
7. Bit fields cannot be arrayed.
8. Bit fields should be assigned values that are within the range of their size. If we try to assign larger values, behaviour would be unpredicted.

Scanned by CamScanner

Files and streams :

The C I/O system provides a level of abstraction between the programmer and the device.

This abstraction is called a stream and the actual device is called files.

The C file system is designed to work with wide variety of devices i.e terminals, disk drives and tape drives.

Even though each device is very different, the buffered file system transforms each into a logical device called streams.

There are two types of streams: text and binary.

Text streams :

A text stream is a sequence of characters. A text stream is organized into lines terminated by newline character.

Binary streams :

A binary stream is a sequence of bytes that has one-to-one correspondence to the bytes in the device.

Files :

In C, a file may be anything from a disk file to terminal. We can associate a stream with a specific file by performing open operation.

Once a file is open, information can be exchanged between files and program.

Scanned by CamScanner

File Management in C

Until now we have been using console oriented I/O operation. Console oriented I/O use the terminal (keyboard and screen) as the target place.

The console oriented I/O operation pose two major problems:
- It is difficult and time consuming to handle large amount of data through terminals.
- The entire data is lost when either the program is terminated or the computer is turned off.

So files are used to store information peomanently in the secondary memory such as disks.

C seepports a number of functions to perform basic file operation such as:
- naming a file
- opening a file
- reading and writing data into files
- closing a file.

Function Name :	Operation :
fopen() :	opens a file
fclose() :	Closes a file
putc() :	writes a character to a file
fputc() :	Same as putc()
getc() :	Reads a character from a file
fgetc() :	Same as getc()
fgets() :	Reads a string from a file
fputs() :	Writes a string to a file.

Scanned by CamScanner

fseek(): seeks to a specific byte in a file.
ftell(): Returns the current file position
fprintf(): Writes a set of data values to a file.
fscanf(): Reads a set of data values from a file.
feof(): Returns true if end-of-file is reached.
ferror(): Returns true if an error has occured.
frewind(): Resets the file position indicator to the beginning
remove(): Erases a file
fflush(): Flushes a file.

Defining and opening a file:

To define a file we have to specify certain things about the file to the operating system. They include filename, Data structure and purpose.

Filename is a pointer to string of characters that make up a valid filename and may include a path specification.

Data structure of a file is defined in a control structure called FILE in the stdio.h.

When we open a file, we must specify what we want to do with the file.

The general format for declaring and opening a file is:

```
FILE *fp;
fp = fopen ("filename", "mode");
```

Here fp is a pointer to type FILE. FILE is a structure defined in I/O library. This pointer will contain all the information about file and used as a communication link between the system and the program.

Scanned by CamScanner

The second statement opens a file with the filename and assigns the identifier to FILE pointer fp.

The second statement also specifies the purpose of opening the file. mode is used for this job.

Mode	Meaning
r:	Open a text file for reading
w:	Create a text file for writing
a:	Append to a text file
rb:	Open a binary file for reading
wb:	Create a binary file for writing
ab:	Append to a binary file
r+:	Open a text file for read/write
w+:	Create a text file for read/write
a+:	Append or create a text file for read/write
r+b:	Open a binary file for read/write
w+b:	Create a binary file for read/write
a+b:	Append or create a binary file for read/write

Both the filename and mode are specified as strings. They should be enclosed in double quote marks.

When the mode is "writing", a file with the specified name is created if the file doesnot exist. The content are deleted, if the file is already exists.

When the purpose is "appending" the file is opened with the current contents safe. If the file doesnot exist a file with specified name will be created.

In case of "reading" if the file exists, then the file is opened for reading with contents are safe and if file doesnot exists then it is an error.

Scanned by CamScanner

closing a file :

A file must be closed after all operation is performed. ~~after~~. This ensures that all outstanding information associated with the file is flushed out from the buffer and all links to the file are broken.

```
fclose (file_pointer);
```

Example :

```
...
...
FILE *P1, *P2;
P1 = fopen ("Input.txt", "w");
P2 = fopen ("Output.txt", "r");
...
...
fclose (P1);
fclose (P2);
...
...
```

Input/Output operations on files :

Once a file is opened, reading or writing is performed by using the standard I/O library functions.

The getc and putc functions :

Assume that a file is opened with "w" mode and file pointer is fp1 :

```
putc (c, fp1);
```

writes the character contained in the character variable c to the file associated with fp1.

Similarly `c = getc(fp2);`

would read a character from the file whose pointer is fp2.

The end of file data is indicated by an EOF character which is Control-Z / Control-D in System.

Scanned by CamScanner

Example :

```
#include<stdio.h>
void main()
{
    FILE *f1;
    char c;
    printf("Enter input data");
    f1 = fopen("Input.txt","w");
    while((c = getchar()) != EOF)
    {
        putc(c, f1);
    }
    fclose(f1);
    prntf("output data");
    f1 = fopen("Input.txt","r");
    while(c=getc(f1)) != EOF)
    {
        prntf("%c", c);
    }
    fclose(f1);
}
```

The getw and putw functions :

* The getw and putw are integer oriented functions. These functions are useful when we deal with only integer data.

```
putw(integer, fp);
getw(fp);
```

Scanned by CamScanner

Example :

```
#include <stdio.h>
void main()
{
        FILE *fp;
        int i;
        fp = fopen("Data.txt", "w");
        for (i=1; i<=30; i++)
        {
            putw(i, fp);
        }
        fclose(fp);
        fp = fopen("Data.txt", "r");
        while ((i = getw(fp)) != EOF)
        {
            printf("%d", i);
        }
        fclose(fp);
}
```

The fprintf and fscanf functions :

* The fprintf and fscanf are similar to printf and scanf that can ~~~~ work on files.

* The general form of fprintf and fscanf is

```
fprintf(fp, "control string", list);
fscanf(fp, "control string", list);
```

Scanned by CamScanner

Example:

```
#include<stdio.h>
#include<io.h>
#include<stdlib.h>
int main()
{
    FILE *fp;
    char s[80];
    int t;
    fp = fopen("Example.txt","w");
    printf("Enter a string and number:\n");
    fscanf(stdin,"%s%d",s,&t);
    fprintf(fp,"%s %d",s,t);
    fclose(fp);
    fp = fopen("Example.txt","r");
    fscanf(fp,"%s %d",s,&t);
    fprintf(stdout,"%s %d",s,t);
    return 0;
}
```

Error Handling in I/O operations :

It is possible that an error may occur during I/O operation on a file. Some situation are:

1. Trying to read beyond end-of-file mark.
2. Device overflow.
3. Trying to use a file that has not been opened.
4. Opening a file with invalid filename.
5. Attempting to write to a write protected file.

Scanned by CamScanner

We have two status inquiry library function; feof() and ferror() that can help us detect I/O errors in the files.

If fp is a pointer to file that has just been opened for reading then:

```
if (feof(fp))
    printf("End of data.\n");
```

will display "End of data" on reaching end of the file.

The ferror function reports the status of the file indicated.

```
if (ferror(fp) != 0)
    printf("An error has occured.\n");
```

Whenever a file is opened using fopen, a file pointer is returned. If the file cannot be opened for some reason, then the function returns NULL pointer.

```
if (fp == NULL)
    printf("File could not found");
```

Example:

```
#include<stdlib.h>
#include<stdio.h>
void main()
{
    FILE *fp1;
    int i, number;
    if ((fp1 = fopen("Example.txt", "w")) == NULL)
    {   printf("Cannot open file\n");
        exit(1);
    }
    for (i=0; i<=100; i+=10)
    {   putw(i, fp1);
    }
```

Scanned by CamScanner

```
fclose (fp1);
if ( (fp1 = fopen("Example.txt", "r")) == NULL)
{   printf (" Cannot open file \n");
    exit (1);
}
for ( i=1 ; i<=20; i++)
{   number = getw (fp1);
    if ( feof (fp1))
    {  printf ("\n Ran out of data.\n");
       break;
    }
    else
       printf (" %d \n", number );
}
fclose (fp1);
}
```

Random Access to files :

When we are interested in accessing only particular part of a file, not reading other parts then we can take help of functions fseek, ftell and rewind.

ftell() takes a file pointer and return a number of type long, that corresponds to the current position.

```
long n= ftell (fp);
```

n will give the relative offset of the current position.

rewind() takes a file pointer and reset the position to start of the file.

```
rewind (fp);
n= ftell (fp);
```

Scanned by CamScanner

The statement will assign 0 to n because the file position has been set to start of the file by rewind.

fseek() function is used to move the file position to a desired location within the file.

fseek(file-pointer, offset, position);

Here "offset" is a number or variable of type long.
"position" is an integer number and it can take three of the following values:

0 - Beginning of file
1 - Current position
2 - End of file.

The offset may be positive, meaning move forward and negative meaning move backwards.

operation of fseek function:

fseek(fp, 0L, 0) : Go to beginning

fseek(fp, 0L, 1) : Stay at current position

fseek(fp, 0L, 2) : Go to the end of file.

fseek(fp, m, 0) : Move to (m+1)th byte in the file.

fseek(fp, m, 1) : Go forward by m bytes

fseek(fp, -m, 1) : Go backward m bytes from the current position.

fseek(fp, -m, 2) : Go backward by m bytes from the end.

Scanned by CamScanner

- When the operation is successful, fseek() returns a Zero. If we attempt to move the file pointer beyond the file boundaries, an error occurs and fseek returns -1.

Example:

```
#include <stdio.h>
Void main()
{
    FILE *fp;
    long n;
    char c;
    fp = fopen("Random.txt", "w");
    while ((c=getchar()) != EOF)
        putc(c, fp);
    printf("No. of character entered = %ld\n", ftell(fp));
    fclose(fp);
    fp = fopen("Random.txt", "r");
    n = 0L;
    while ( feof(fp)==0)
    {
        fseek(fp, n, 0); // position to (n+1)th character
        printf("Position of %c is %ld\n", getc(fp),
        n = n+5L;                    ftell(fp));
    }
    putchar('\n');
    fseek(fp, -1L, 2); // position to last character
    do {
        putchar(getc(fp));
    } while (!fseek(fp, -2L, 1));
    fclose(fp);
}
```

Scanned by CamScanner